

Resumen de cierre — Automatizacion de pruebas E2E de autorizacion

Fase 1 · Policy Enforcement Point (custodia-pep)

Fecha de cierre: 17 de septiembre de 2026 • Repositorio: diego1-avendano3/ProyectoCustodio • PR #1 (feat/authorization-e2e-tests → main) • Commit de cierre: 8b214b9

1. Resumen ejecutivo

La matriz de autorizacion de cinco escenarios que hasta ahora se validaba manualmente con **curl** quedo convertida en una suite de pruebas de integracion automatizadas (**AuthorizationE2EIT**), ejecutada contra un motor de politicas OPA real (via Testcontainers) y un mock del servicio financiero (via WireMock). La suite corre en cada Pull Request y en cada push a **main** a traves de GitHub Actions.

El hito quedo formalmente cerrado: el Pull Request #1 fue revisado, validado en CI y mergeado a **main** mediante squash-merge (commit **8b214b9**), con los dos checks de CI en verde tanto en el PR como en el push resultante sobre **main**.

El trabajo tomo considerablemente mas esfuerzo del previsto no por la logica de negocio en si — que resulto correcta desde el principio — sino por una cadena de incompatibilidades de infraestructura de build (Docker Desktop, resolucion de versiones de Testcontainers, y una configuracion faltante de Maven Failsafe) que se documentan en la seccion 4.

2. Alcance implementado

Se creo la clase **AuthorizationE2EIT.java** en **custodia-pep/src/test/java/com/custodia/pep/**, con las siguientes piezas de infraestructura de prueba:

- Un contenedor real de **OPA** (Testcontainers, imagen *openpolicyagent/opa:latest*) que carga las politicas Rego reales del directorio **policies/** del repositorio — no un mock de la decision de autorizacion.
- Un servidor **WireMock** que sustituye a *finance-api-mock*, sirviendo el saldo de la cuenta de prueba *acc-1001*.
- La aplicacion Spring Boot completa levantada con **@SpringBootTest(webEnvironment = RANDOM_PORT)** y **WebTestClient** para ejercer el endpoint real */accounts/{id}/balance* de punta a punta.
- Cinco pruebas (**@Test**), una por cada escenario de la matriz de autorizacion original de Fase 1 (ver tabla siguiente).

2.1 Matriz de escenarios automatizados

#	Escenario	Headers / parametros	Resultado esperado
1	Owner, riesgo bajo	X-Employee-Id: emp-42 X-Role: employee X-Risk-Score: 10	200 OK — saldo completo

2	No-owner	X-Employee-Id: emp-1 X-Role: employee X-Risk-Score: 10	403 Forbidden — access_denied
3	Owner, riesgo elevado	X-Employee-Id: emp-42 X-Role: employee X-Risk-Score: 80	200 OK — numero de cuenta enmascarado
4	Analista de fraude, sin caso abierto	X-Employee-Id: emp-99 X-Role: fraud_analyst X-Risk-Score: 10	403 Forbidden
5	Analista de fraude, con caso abierto	X-Employee-Id: emp-99 X-Role: fraud_analyst ?caseOpen=true	200 OK — numero de cuenta enmascarado

Los nombres de headers y el parametro **caseOpen** fueron verificados contra los ejemplos de *curl* documentados en el README del repositorio y contra los campos de entrada (*input.subject.**, *input.context.case_open*) usados por la politica Rego real (**policies/authz.rego**), confirmando que la suite automatizada reproduce fielmente la validacion manual original.

3. Arquitectura de la prueba

A diferencia de una prueba unitaria con mocks, esta suite ejercita la cadena de decision real: **PEP (Spring WebFlux) → PDP (OPA, politica Rego real) → recurso protegido (WireMock simulando finance-api)**. Esto da una garantia mucho mas fuerte de que el comportamiento de autorizacion observado en produccion coincide con el validado en CI, al costo de una infraestructura de prueba mas compleja (contenedores Docker en tiempo de test).

El ciclo de vida de ambos dobles de prueba (OPA y WireMock) se maneja manualmente mediante un bloque estatico, en lugar de la extension JUnit 5 de Testcontainers (**@Testcontainers / @Container**). Esta decision se tomo durante la depuracion (seccion 4) y queda registrada como punto abierto en la seccion 5.

4. Incidentes tecnicos y causas raiz

Llevar la suite de "compila" a "pasa en CI de forma confiable" involucro resolver tres problemas independientes, en cascada. Se listan en el orden en que se enfrentaron.

Problema	Causa raiz	Resolucion
Testcontainers no podia conectar a Docker en el equipo Windows local (named pipe y TCP fallaban con BadRequestException).	Incompatibilidad conocida, no resuelta en upstream, entre Docker Engine 29.x / Docker Desktop 4.89 y la libreria docker-java usada por Testcontainers (testcontainers-java #11422, #11419).	No se intento parchear localmente. Se desplazo toda la validacion de Testcontainers a GitHub Actions (ubuntu-latest, Docker 28.0.4), que no presenta el problema.
La version de Testcontainers resuelta en el build era 1.19.8 en vez de la 1.20.4 esperada.	El BOM de spring-boot-dependencies se importaba antes que el BOM propio de testcontainers-bom en el pom.xml raiz; Maven aplica la regla "la primera importacion de BOM gana".	Se fijo <code><version>\${testcontainers.version}</version></code> explicitamente en las dependencias de Testcontainers de custodia-pep/pom.xml (una version declarada de forma directa siempre gana sobre la gestionada por un BOM).
Fallo persistente y determinista: <i>Failed to find merged annotation for @BootstrapWith(...)</i> en todo test <i>*IT.java</i> , independientemente de su contenido.	maven-failsafe-plugin, al no heredar de spring-boot-starter-parent, no apunta por defecto a target/classes. Como Failsafe corre en la fase integration-test (despues de package/repackage), terminaba incluyendo el JAR reempaquetado por Spring Boot (con las clases anidadas bajo BOOT-INF/classes/...) en el classpath de test, lo que corrompia la resolucion de metadatos de anotaciones de Spring.	Se agrego <code><classesDirectory>\${project.build.outputDirectory}</classesDirectory></code> a la configuracion de maven-failsafe-plugin en custodia-pep/pom.xml. Confirmado como la causa raiz real: tras el cambio, el mismo commit que antes fallaba paso de forma consistente en CI.

El tercer problema fue, por lejos, el mas costoso de diagnosticar: se descarto primero la hipotesis de que `@Testcontainers/@Container` chocaba con la resolucion de `@BootstrapWith` de Spring, mediante clases de prueba desechables ("smoke tests") para aislar variables. Esa hipotesis resulto ser una pista falsa: el fallo ocurría en 0.1 segundos, antes de que arrancara cualquier contenedor, lo que finalmente apunto hacia la fase de empaquetado de Maven en lugar del contenido de la prueba en si.

5. Resultado final y pendientes

Estado en CI: ambos checks ("CI / Build & test (Java 21 / Maven)" y "CI / Politicas Rego (Custodia PDP)") en verde, tanto en el ultimo push del PR #1 (commit 3ada12d) como en el push de merge a main (commit 8b214b9, ejecucion CI #12).

- **Manejo manual de contenedores vs. `@Testcontainers/@Container`:** no se probó en aislamiento si el fix de `classesDirectory` por si solo habria bastado sin tambien remover la extension JUnit 5 de Testcontainers. Es un posible cleanup de estilo, no bloqueante.
- **Cobertura de limites de riesgo:** la politica permite ver el saldo propio con riesgo <70 (completo) y 70–89 (enmascarado); el comportamiento para `risk_score ≥ 90` no esta cubierto por ninguno de los 5 escenarios (tampoco lo estaba en la validacion manual original).

- **Rama de feature:** feat/authorization-e2e-tests fue eliminada tanto local como remotamente tras el merge; no quedan ramas huérfanas.

6. Proximos pasos sugeridos

Con la Fase 1 cerrada y verificada de extremo a extremo, las líneas de trabajo abiertas para continuar son:

- **Fase 2 — Sensitive Data Gateway + AWS KMS:** siguiente bloque funcional del roadmap.
- **Mutation testing (PIT):** medir la calidad real de la suite de pruebas mas alla de la cobertura de líneas.
- **SAST / SCA:** analisis estatico de seguridad y de dependencias.
- **requirements.md en formato EARS:** formalizar los requisitos antes de seguir construyendo funcionalidad nueva.

Documento generado como cierre de hito — Proyecto Custodia. Complementa a docs/reports/fase1-resumen.pdf (resumen funcional de Fase 0-1) con el detalle específico de la automatización de pruebas de autorización y su validación en CI.